

Caleb Acree

XTEA Project

3/16/2026

XTEA is a 64-bit block cipher that splits its input into halves. It uses a 128-bit key that is stored as four 32-bit words and encrypts the two split halves. The cipher runs for 32 cycles, first on one half of the data and using the other half of the data to do so. Then it uses the now modified half of data to modify the other half. That is one loop. So, while the loop only runs 32 times, the data is modified 64 times.

The constant delta is gotten from the golden ratio. It has no special structure or flaws for attackers to exploit. It can be called “nothing-up-my-sleeve-number.” The sum variable starts are zero and accumulates one delta per cycle. It acts as a round counter. Since the value of the variable sum is different ever cycle the same keywords get mixed in differently each time even though the key itself never changes. Each cycle it selects one of four key words for each of the two updates. The first update (before $\text{sum} += \text{delta}$) uses $\text{sum} \& 3$, the two least-significant bits of the current sum. The second update (after $\text{sum} += \text{delta}$) uses $(\text{sum} \gg 11) \& 3$, pulling two bits from a higher position of the now-incremented sum. The 11th bit was chosen because it ensures that the two selectors in the same cycle are never equal.

Identify the signal names we will use in Quartus:

Clock:

CLOCK_50: PIN_N2

PS/2:

PS2_KBCLK: PIN_C24

PS2_KBDAT: PIN_D26

LCD:

LCD_ON: PIN_L4

LCD_BLON: PIN_K2

LCD_EN: PIN_K3

LCD_RS: PIN_K1

LCD_RW: PIN_K4

LCD_DATA[7..0]:

0: PIN_J1

1: PIN_J2

2: PIN_H1

3: PIN_H2

4: PIN_J4

5: PIN_J3

6: PIN_H4

7: PIN_H3

It creates a dummy environment around the actual XTEA circuit. It provides the inputs it needs and then monitors the outputs. The line always ``#5 clk=~clk`` just generates a clock signal that flips every 5 nanoseconds, creating a 10ns clock period. It does this so it is actually able to advance through its rounds. The next part takes three arguments: a key K, a plaintext P, and a known-correct ciphertext C. It then does the next two things in sequence. First it feeds the circuit K and P with `enc=1` (encrypt), waits for `done` to go high, and then checks whether the output matches C. If it doesn't, it prints an error message and calls ``$stop``. This halts the simulation. After the halt is resolved it feeds K and C back in with `enc=0` (decrypt), waits for the `done` signal. Lastly it checks whether the output matches the original P. So, every vector tests both directions; both have to be correct to pass. The initial block holds the circuit in reset for 5 clock cycles. Then releases it and runs ``run_vec`` four times with the pre-computed key/plaintext/ciphertext combinations. If the circuit creates matching outputs, it's implemented correctly. Failure is indicated by the ``$stop`` printing a message. Success is indicated by reaching the last line ``$display("All XTEA tests passed.")``. Lastly followed by ``$finish``, which happens if all four ``run_vec`` calls completed without encountering an error.

ESD: Touch a grounded metal surface before or use a anti-static wrist band while handling the board.

Power-up Sequence: Connect all cables while the board is unpowered. Power on the FPGA board. Upload project from quartus.

Cable handling: connect and disconnect PS2 cables while board power is off. Otherwise, it can damage the board.